CampusConnect: A Role-Based, Real-Time Web Platform for Unifying Academic Workflows and Campus Communication

Aditya Gour, Aman Kumar Chauhan, Anjali Kumari, Aayushman Pathak, Akshat Verma

Department of Computer Science and Engineering
Arya College of Engineering and IT, Kukas, Jaipur, Rajasthan, India
Guided by Er. Ram Babu Buri, Assistant Professor, Department of CSE
Corresponding author: [e-mail address]

***Abstract*—Students and faculty in many colleges coordinate coursework, notices, club activities, and informal communication through a mix of learning platforms, messaging groups, notice boards, and paper. This fragmentation scatters information and leaves access control to whoever administers each channel. This paper presents CampusConnect, a full-stack web application that brings academic workflows (subjects, assignments, submissions, and grading), announcements, clubs and events, a moderated community board, and one-to-one real-time chat into a single platform for students, teachers, and administrators. The system is a layered modular monolith built with React and Vite on the client, Spring Boot on Java 21 on the server, and MySQL for persistence, with stateless JSON Web Token (JWT) authentication for both REST and STOMP-over-WebSocket traffic. Its central design principle is that every authorization and consistency rule is enforced in server-side code: role checks are combined with ownership and audience matching, event capacity is protected by pessimistic locking, chat delivery is idempotent, and every real-time subscription is authorized. The implementation is verified by 227 automated unit and controller tests across 26 test classes, all of which passed in the most recent recorded run. No performance, usability, or security-audit measurements are claimed; instead, the paper specifies a reproducible evaluation protocol and states the current limitations explicitly.**

***Index Terms*—Campus information systems, learning management systems, role-based access control, JSON Web Token, WebSocket, STOMP, Spring Boot, React, academic workflow management.**

I. Introduction

Higher-education institutions depend on two kinds of digital support: structured academic administration (courses, assessments, deadlines, and grades) and less structured campus communication (announcements, extracurricular activity, peer discussion, and direct contact between students and teachers). Learning management systems (LMSs) address much of the first category [1], [2], and smart-campus initiatives aim to integrate institutional services more broadly [3]. In practice, many colleges still spread these functions across an LMS or spreadsheet for coursework, instant-messaging groups for notices, physical notice boards for events, and personal messaging for queries. Each channel has its own membership model, and none gives the institution a consistent view of who may see or change what.

CampusConnect was developed as a B.Tech. group project at Arya College of Engineering and IT to combine these functions in one application with one identity model. It is deliberately scoped to a single institution: registration accepts one configured college. This paper reports the system's architecture, design decisions, verification evidence, and the evaluation work that remains. It makes no claim of pedagogical impact or of performance at scale.

The main contributions are: (1) a modular architecture that delivers six functional modules through one REST API, one WebSocket endpoint, and one relational schema; (2) a centralized authorization design that combines role checks with resource ownership and student-audience matching; (3) explicit consistency mechanisms for shared resources, namely pessimistic locking, database uniqueness, idempotent messaging, and transaction-synchronized file storage; and (4) a transparent verification record and evaluation protocol that distinguishes implemented, tested, and still-unmeasured properties.

II. Related Work

A. Learning Management and Smart Campus Platforms

Watson and Watson distinguish learning management systems, which manage learning across an organization, from course management systems, which focus on individual courses [1]. Moodle, a widely studied open-source course management system, was grounded in social-constructionist pedagogy and developed with its user community [2]. Such platforms provide mature assignment and grading workflows, but their scope is usually the course: campus announcements, clubs, event registration, and informal discussion sit outside that boundary or require separate plug-ins. Min-Allah and Alrashed describe the smart campus as the integration of institutional services, infrastructure, and data [3]. CampusConnect addresses a narrow, software-only slice of that vision.

B. Online Community and Communication in Education

The Community of Inquiry framework identifies social, cognitive, and teaching presence as interdependent elements of effective online education [4], and Rovai developed a

validated instrument for measuring classroom community [5]. These works motivate placing discussion and messaging alongside academic workflows rather than on unmanaged channels, and they supply candidate instruments for future evaluation. CampusConnect does not yet test any pedagogical hypothesis.

C. Web Architecture and Real-Time Communication

REST defines constraints such as client–server separation, statelessness, and a uniform interface [6]. Fowler's layered architecture, service layer, and data transfer object patterns separate business logic from presentation and persistence [7]. Microservices offer independent scaling and deployment but add distribution and operational complexity [8]; for a small team and one institution, a modular monolith with clear internal boundaries is a defensible alternative. The WebSocket protocol provides full-duplex communication over one TCP connection [14], and STOMP adds destinations, subscriptions, and message frames on top of it [15]. Spring supports STOMP with an in-memory broker, user destinations, and channel interceptors [16]. Because the WebSocket handshake and STOMP subscriptions bypass the filters that protect REST endpoints, they must be authorized separately.

D. Access Control and Secure Application Design

Role-based access control (RBAC) assigns permissions to roles rather than to individual users [9], and the NIST model standardizes its core components [10]. Pure RBAC cannot express rules such as "a teacher may grade only submissions for subjects that teacher teaches," which depend on relationships between users and resources. Broken access control ranks first in the OWASP Top 10:2025, which recommends server-side enforcement, deny-by-default, and record-ownership checks [11]. JSON Web Tokens provide a compact, signed format for claims [12], and bcrypt makes the cost of password hashing tunable as hardware improves [13]. Security controls must therefore be tested as part of the system rather than assumed from framework use.

III. Research Gap and Objectives

Existing work provides mature solutions to the individual sub-problems: course management [1], [2], access-control models [9], [10], and real-time messaging protocols [14], [15]. The project-level gap is not a missing algorithm; it is the engineering problem of integrating these sub-problems into one coherent, small-footprint platform without losing consistent authorization, data integrity, or transparency about what has been verified. When academic records, community content, event capacity, and private messages share one identity model, a single authorization mistake can expose data across modules.

- **O1:** Provide one identity and profile model for students, teachers, and administrators, with administrator privileges granted only through trusted server-side configuration.
- **O2:** Support the complete assignment lifecycle, from drafting and publication to submission, revision, grading, and return, with deadlines enforced on the server.
- **O3:** Provide audience-aware announcements, club membership, and capacity-limited event registration that stay correct under concurrent requests.
- **O4:** Provide a moderated community board and authenticated one-to-one real-time chat with online/offline presence.
- **O5:** Enforce authorization in server-side code for every REST and WebSocket path, consistent with OWASP guidance [11].
- **O6:** Back the implementation with automated tests and define a reproducible protocol for performance, usability, and security evaluation.

IV. Proposed System Architecture

CampusConnect follows a client–server architecture with a layered backend. A React 19 single-page application (SPA) communicates with the backend over a JSON REST API under `/api/**` and a STOMP-over-WebSocket endpoint at `/ws/sockjs` with a SockJS fallback. In the Spring Boot backend, the security filter chain validates JWTs before requests reach 12 REST controllers exposing 67 handler methods, while channel interceptors authenticate WebSocket connections and authorize subscriptions before messages reach the STOMP controller. Controllers exchange data transfer objects (DTOs) rather than JPA entities [7]; services hold business rules, authorization, and transaction boundaries; and Spring Data JPA repositories persist 19 entities in MySQL 8. A global exception handler maps domain exceptions to a uniform JSON error body and returns generic messages for unexpected failures. In production, a multi-stage Docker build packages the compiled SPA inside the Spring Boot executable, so one container serves both the website and the API from the same origin.

A. Modules and Interfaces

Module 1—Authentication and profiles (Aman Kumar Chauhan)—implements student and teacher registration, BCrypt password hashing, JWT issuance and validation, profile management, and the shared database and deployment settings. Module 2—Academics (Akshat Verma)—implements subjects, the assignment lifecycle, attachments, submissions, rosters, and grading. Module 3—Announcements (Anjali Kumari)—implements audience- and department-filtered notices with pinning and author-scoped editing. Module 4—Chat, community, and clubs and events (Aayushman Pathak)—implements one-to-one real-time messaging, presence, the moderated community and feedback hub, club membership, and capacity-limited event registration. Module 5—Platform integration (Aditya Gour)—implements the project setup, landing page, role-based dashboard, academics page, college registration flow, security and performance fixes, and Docker deployment.

Representative endpoints are `POST /api/auth/login`, `POST /api/assignments/{id}/publish`, `POST /api/assignments/{id}/submissions`, `POST /api/submissions/{id}/grade`, `POST /api/events/{id}/register`, `POST`

/api/community/posts/{id}/report, GET /api/conversations/{id}/messages, and the STOMP destination /app/chat.send. Every endpoint derives the acting user from the authenticated principal; a caller-supplied user identifier is never treated as proof of authorization.

B. Data Model

MySQL stores 19 tables. User profiles are split into a users table and role-specific student_profiles and teacher_profiles tables with unique enrollment-number and employee-ID constraints. Subjects belong to teachers and contain assignments, which receive submissions; clubs have memberships and organize campus events with registrations; community posts have comments, votes, and reports; and conversations have participants and messages. File metadata (original name, storage key, content type, and size) is embedded in assignments and submissions, and moderation_audit_logs records moderator actions on other users' content.

C. Academic Workflow

Assignments are drafts unless published at creation; publication is one-way and requires a future deadline. Maximum marks cannot be reduced below marks already awarded, and an assignment with submissions cannot be deleted or moved to another subject. Each student submits at most once per assignment, enforced by a service check and a unique constraint on (assignment_id, student_id). After the deadline, a first submission is rejected unless late work is allowed, in which case it is flagged as late; revision is permitted only before the deadline and while ungraded. Marks must lie between zero and the maximum, and grades stay hidden until the teacher returns the work. For teachers, the submissions view is a full roster that joins every eligible student with any submission, so students who have not submitted are visible.

V. Authorization and Consistency Design

A. Layered Authorization

Authorization combines RBAC [9], [10] with relationship checks that pure RBAC cannot express. First, every /api/** route except /api/auth/** requires an authenticated principal. Second, seventeen method-level @PreAuthorize annotations restrict operations by role: only teachers create or grade assignments, only students submit, and only teachers and administrators post announcements. Third, a dedicated AcademicAccessService centralizes ownership and audience rules: a teacher manages only subjects whose teacher_id matches their identity, and a student sees a subject only when their department, and where specified their year, course, and section, match its audience. Other modules apply equivalent checks in their services; for example, only conversation participants may read or send messages, and only authors and administrators may delete community content. Finally, when a user may not see an academic resource, the service responds 404 rather than 403, so the existence of the resource is not revealed.

B. Consistency Mechanisms

Event capacity: registration loads the event row with a JPA PESSIMISTIC_WRITE lock (SELECT … FOR UPDATE) before checking status, deadline, and remaining capacity, which serializes concurrent registrations and prevents overbooking [17]; a unique constraint on (event_id, user_id) prevents duplicates. *Idempotent messaging:* each chat message carries a client-generated UUID (clientMsgId) backed by a unique constraint, so retries after a reconnect do not create duplicate messages, mirroring idempotent HTTP requests [18]. *Votes and reports:* unique constraints allow at most one upvote and one report per user per post. *File storage:* uploads are written under random UUID names and deleted if the surrounding transaction rolls back, and replaced files are removed only after commit, which prevents both orphaned files and dangling references.

VI. Real-Time Chat and Presence

On STOMP CONNECT, an inbound channel interceptor validates the JWT from the native Authorization header and binds the principal to the session. A second interceptor checks every SUBSCRIBE to /topic/conversations/{id} and rejects it unless the principal is a participant, which prevents eavesdropping by guessing identifiers. Because STOMP handlers do not run bean validation, the /app/chat.send handler re-validates the payload (empty content, a missing clientMsgId, or content over 4,000 characters), verifies membership, and persists the message idempotently. The saved message is broadcast to the conversation topic and to each participant's user queue, and errors are sent to the sender's private /user/queue/errors destination so the connection is not dropped.

The client reconnects automatically after 5 s and, when no live connection exists, falls back to a REST send endpoint that uses the same clientMsgId, so deduplication covers both paths. Presence is tracked in memory as the set of active WebSocket sessions per user; a user is online while at least one session is open, and transitions are broadcast on /topic/presence. The in-memory broker and presence registry suit a single instance but would require an external broker relay and shared presence storage to scale horizontally.

VII. Security, Privacy, and Ethical Considerations

Implemented controls include BCrypt password hashing [13]; HMAC-signed, expiring JWTs (24 h by default) whose signing secret has no default, so the application refuses to start without JWT_SECRET [12]; per-request reloading of the user, so role changes take effect immediately; server-side authorization on every REST and WebSocket path [11]; no public administrator registration, since administrators are provisioned only through the server-side ADMIN_PROVISION_EMAIL variable; file-extension allow-listing, a 10 MB limit, generated storage names, and path-traversal checks; generic error messages; and a fail-fast production profile with no fallback credentials.

Code inspection also identified known weaknesses. The JWT is stored in browser localStorage, which OWASP

advises against because any cross-site scripting flaw can read it [19]. There is no token revocation, rate limiting, or e-mail verification; teacher employee IDs are not checked against institutional records; the WebSocket endpoint always allows `localhost` origins; and uploads are not scanned for malware.

The system processes names, e-mail addresses, enrollment numbers, grades, and private messages. Before deployment beyond a classroom demonstration, the institution should define retention and deletion policies, limit administrative access to private messages (none is currently exposed through the API), explain how moderation logs are used, and confirm compliance with applicable data-protection law. Because moderation tools give administrators considerable power over student speech, transparent moderation guidelines are an ethical prerequisite as well as a technical one.

VIII. Evaluation Methodology

The available evidence consists of automated tests. The backend contains 26 JUnit 5 and Mockito test classes with 227 test methods: identity, profiles, and JWT (25); academics (104); announcements (22); clubs and events (15); community (17); and chat, presence, and subscription authorization (44). All passed with zero failures, errors, or skips in the most recent recorded run (11 Oct 2026). Controller tests use MockMvc with the production exception handler, and service tests check business and authorization rules against mocked repositories. These results show that the specified rules behave as intended in isolation, but not against a real MySQL instance or message broker. The frontend has no automated tests, code coverage has not been measured, and no performance, usability, or security-audit measurements or user studies have been conducted. The following protocol, organized by ISO/IEC 25010:2023 [20], is proposed for future work.

- Functional suitability: integration tests against a disposable MySQL 8 instance covering the full assignment lifecycle, with pass rate per requirement and traceability to the SRS.
- Security: an authorization matrix that calls all 67 REST handlers and the STOMP destinations as an anonymous user, a student, the owning teacher, a non-owning teacher, and an administrator, with zero policy deviations as the release criterion, plus dependency and dynamic scanning.
- Reliability: $N = 100$ simultaneous registrations against an event of capacity $k < N$ over at least 30 trials (expected overbookings: zero), and replay of duplicate `clientMsgId` values across reconnects (expected duplicates: zero).
- Performance: median and 95th-percentile latency, throughput, and error rate for representative endpoints at increasing numbers of virtual users on documented hardware, plus send-to-receive chat latency.
- Usability: task-based sessions with consenting students and teachers measuring completion rate and time on task, followed by the System Usability Scale [21].

Results should be reported with sample sizes, environment details, and confidence intervals where the sample size permits.

IX. Implementation and Deployment

The implemented stack is React 19, React Router, Vite, Tailwind CSS 4, Axios, and `@stomp/stompjs` with SockJS on the client; Java 21, Spring Boot 4.1 (Web MVC, Data JPA, Security, Validation, and WebSocket), jjwt, and Maven on the server; MySQL 8 with Hibernate and HikariCP for persistence; and JUnit 5, Mockito, and MockMvc for testing. Authenticated pages are lazy-loaded inside a shared dashboard layout, and client-side route guards are a usability convenience rather than a security boundary.

The team built the system incrementally using a feature-branch workflow with pull requests into a protected `main` branch; the history contains 76 commits, including 21 merged pull requests. Integration proceeded from authentication and profiles to the dashboard shell, chat, community, announcements, academics, and clubs and events, followed by security and performance fixes. All environment-specific values come from environment variables, and a separate production profile fails at startup if credentials are missing. The production image is a multi-stage Docker build (Node 22, Maven with Temurin 21, and a Temurin 21 JRE runtime).

X. Limitations

No performance, usability, or security-audit measurements exist, and there are no integration or frontend tests. Tokens are stored in `localStorage` without revocation or rate limiting, and teacher accounts are unverified. The in-memory broker, presence registry, and local-disk file storage confine the system to a single instance. Role-specific dashboard widgets currently use labelled mock data, with only announcements fetched live, although the dedicated module pages use the live API throughout. Chat is one-to-one only, unread counts are tracked only on the client, there are no notifications, and registration is restricted to one institution. The schema is maintained by Hibernate's update mode rather than versioned migrations. CampusConnect should therefore be regarded as a working, unit-tested prototype whose institutional value remains to be established empirically.

XI. Conclusion and Future Work

This paper presented CampusConnect, a role-based web platform that integrates academic workflows, announcements, clubs and events, a moderated community board, and real-time chat for students, teachers, and administrators. Its main engineering contributions are a centralized, relationship-aware authorization design; explicit consistency mechanisms for shared resources; and an authenticated real-time channel that authorizes both the connection and each subscription. Future work should run the evaluation protocol of Section VIII and publish the results; move tokens to HTTP-only cookies with refresh-token rotation and add rate limiting, e-mail verification, and administrator approval of teacher accounts; connect dashboards to live aggregate endpoints and add notifications; adopt versioned schema migrations; introduce an external broker relay and object storage for multi-instance deployment; and generalize the single-college constraint into multi-institution tenancy.

References

[1] W. R. Watson and S. L. Watson, "An argument for clarity: What are learning management systems, what are they not, and what should they become?" *TechTrends*, vol. 51, no. 2, pp. 28–34, 2007.

[2] M. Dougiamas and P. C. Taylor, "Moodle: Using learning communities to create an open source course management system," in *Proc. EdMedia*, Honolulu, HI, USA, 2003, pp. 171–178.

[3] N. Min-Allah and S. Alrashed, "Smart campus—A sketch," *Sustainable Cities and Society*, vol. 59, Art. no. 102231, 2020.

[4] D. R. Garrison, T. Anderson, and W. Archer, "Critical inquiry in a text-based environment: Computer conferencing in higher education," *The Internet and Higher Education*, vol. 2, no. 2–3, pp. 87–105, 1999.

[5] A. P. Rovai, "Development of an instrument to measure classroom community," *The Internet and Higher Education*, vol. 5, no. 3, pp. 197–211, 2002.

[6] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. California, Irvine, CA, USA, 2000.

[7] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

[8] N. Dragoni *et al.*, "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*. Cham, Switzerland: Springer, 2017, pp. 195–216.

[9] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[10] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Inf. Syst. Secur.*, vol. 4, no. 3, pp. 224–274, 2001.

[11] OWASP Foundation, "OWASP Top 10:2025—A01:2025 Broken Access Control," 2025. [Online]. Available: https://owasp.org/Top10/2025/

[12] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015.

[13] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf. (FREENIX Track)*, 1999.

[14] I. Fette and A. Melnikov, "The WebSocket protocol," RFC 6455, IETF, Dec. 2011.

[15] "STOMP protocol specification, version 1.2." [Online]. Available: https://stomp.github.io/stomp-specification-1.2.html

[16] Spring Framework Reference Documentation, "WebSockets: STOMP." [Online]. Available: https://docs.spring.io/spring-framework/reference/web/websocket/stomp.html

[17] P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Reading, MA, USA: Addison-Wesley, 1987.

[18] R. Fielding, M. Nottingham, and J. Reschke, Eds., "HTTP semantics," RFC 9110, IETF, Jun. 2022.

[19] OWASP Cheat Sheet Series, "HTML5 security cheat sheet—Local storage." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html

[20] *Systems and Software Engineering—SQuaRE—Product Quality Model*, ISO/IEC 25010:2023, 2023.

[21] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*, P. W. Jordan *et al.*, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.

Appendix A
Implementation Checklist

- Derive the acting user only from the authenticated principal; never trust user identifiers in request bodies.
- Enforce role, ownership, and audience checks in services, and return 404 for hidden academic resources.
- Authenticate every STOMP `CONNECT` and authorize every `SUBSCRIBE` to a conversation topic.
- Back every "at most one" rule with a database unique constraint.
- Lock the event row before checking and incrementing registration capacity.
- Require a client message ID for every chat message and deduplicate on it.
- Tie file writes and deletions to the surrounding database transaction.
- Refuse to start in production without a JWT secret and database credentials.
- Run the authorization matrix and concurrency tests before any institutional deployment.